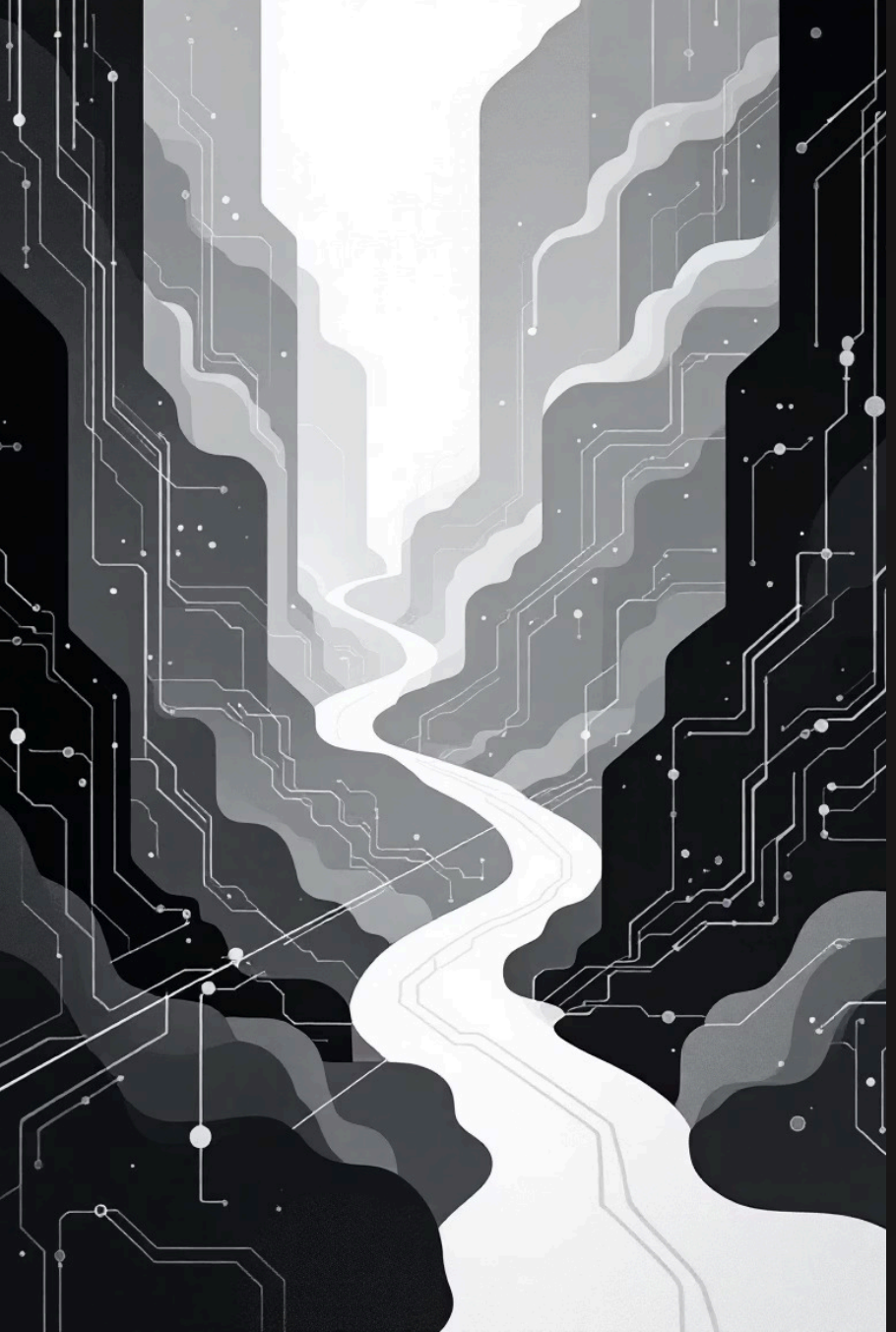
Design Patterns Estruturais Aplicados ao Varejo de Bebidas

Integrantes: Davi Silva Amâncio, Gustavo Monteiro Fonseca e Hugo Galvão Figueiredo Neves

Made with GAMMA



Categoria Estrutural

Composição e organização de classes e objetos

A categoria estrutural trata de **como combinar objetos e classes** para formar estruturas maiores e flexíveis.

O foco está em **adaptabilidade**, **unificação de interfaces**, **ocultação de complexidade** e **hierarquias parte-todo**.

No contexto do varejo de bebidas, a arquitetura deve suportar **estoques**, **pedidos**, **transações**, **produtos com categorias** e **consultas de alto volume** — cenário ideal para padrões estruturais.

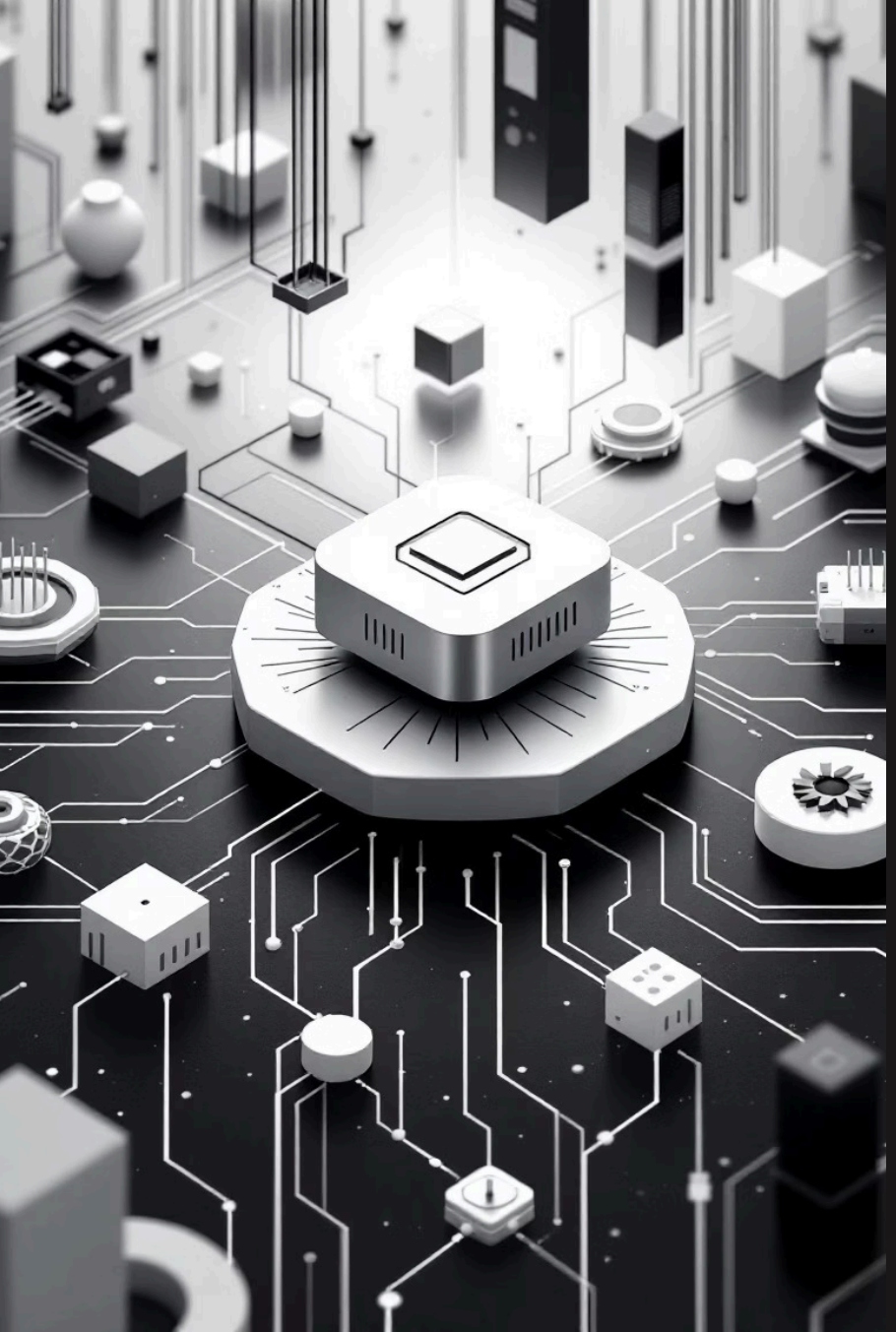
Made with GAMMA

Visão Geral do Sistema (Objeto de Estudo)

Ponto de venda de varejo de bebidas (BD NoSQL)

- O sistema lida com **produtos**, **pontos de venda**, **estoques**, **pedidos**, **itens de pedido**, **pagamentos** e **relatórios**.
- Alto volume de operações em **estoque**, **vendas** e **consultas**.
- Diversas estruturas internas (produtos, fabricantes, categorias, pedidos) interagem entre si.
- Essa dinâmica favorece o uso dos padrões escolhidos para reduzir acoplamento, unificar interfaces e organizar conjuntos de objetos complexos.

Made with GAMMA



Adapter

Problema e Objetivo

Problema

Sistemas de PDV recebem dados de fontes diferentes (fornecedores, APIs, formatos variados como CSV, XML, JSON e coleções MongoDB com formatos distintos). As interfaces dessas fontes não combinam com a interface esperada pelo sistema interno.

Objetivo

Converter uma interface existente **para a interface que o sistema espera**, sem modificar o código original da fonte de dados, garantindo a integração suave de sistemas heterogêneos.

Made with GAMMA

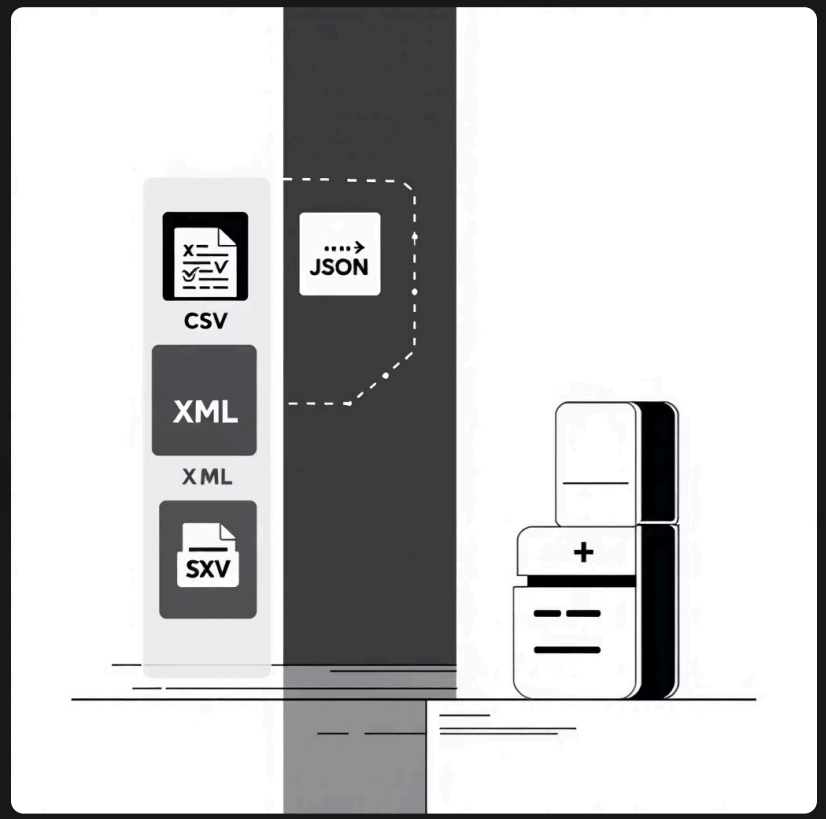
Adapter

Exemplo aplicado ao varejo de bebidas

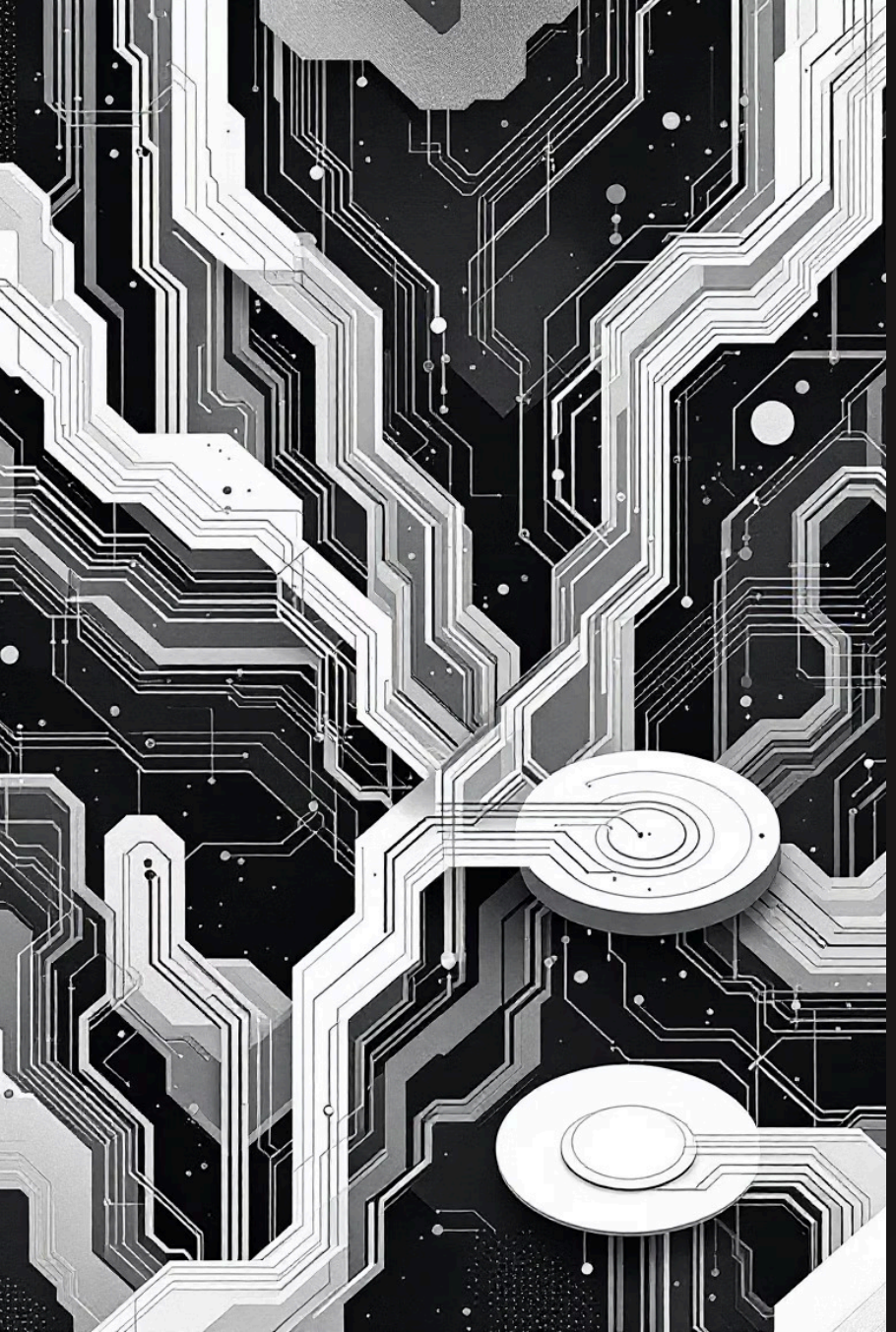
Contexto: O PDV importa produtos de fornecedores em formatos variados (CSV, XML, JSON), enquanto o sistema interno espera um objeto `ProdutoPadronizado`.

Solução: Criar adaptadores como `CSVProdutoAdapter`, `APIFornecedorAdapter` ou `MongoProdutoAdapter` que transformam o formato original no formato interno, implementando a interface comum `IProduto`.

Benefício: O restante do sistema não precisa saber de onde veio o dado — ele recebe sempre a mesma interface, simplificando cadastros, atualizações de estoque e consultas, e promovendo um acoplamento baixo.



Made with GAMMA



Facade

Problema e Objetivo

Problema

O PDV possui múltiplos subsistemas complexos: estoque, pagamentos, relatórios, pedidos, autenticação, alertas e categorias. Cada subsistema tem operações próprias, tornando o uso direto complexo e propenso a erros.

Objetivo

Fornecer uma **interface unificada e simplificada** para operações comuns, reduzindo a complexidade de interação e o acoplamento entre a aplicação cliente e os subsistemas internos.

Made with GAMMA

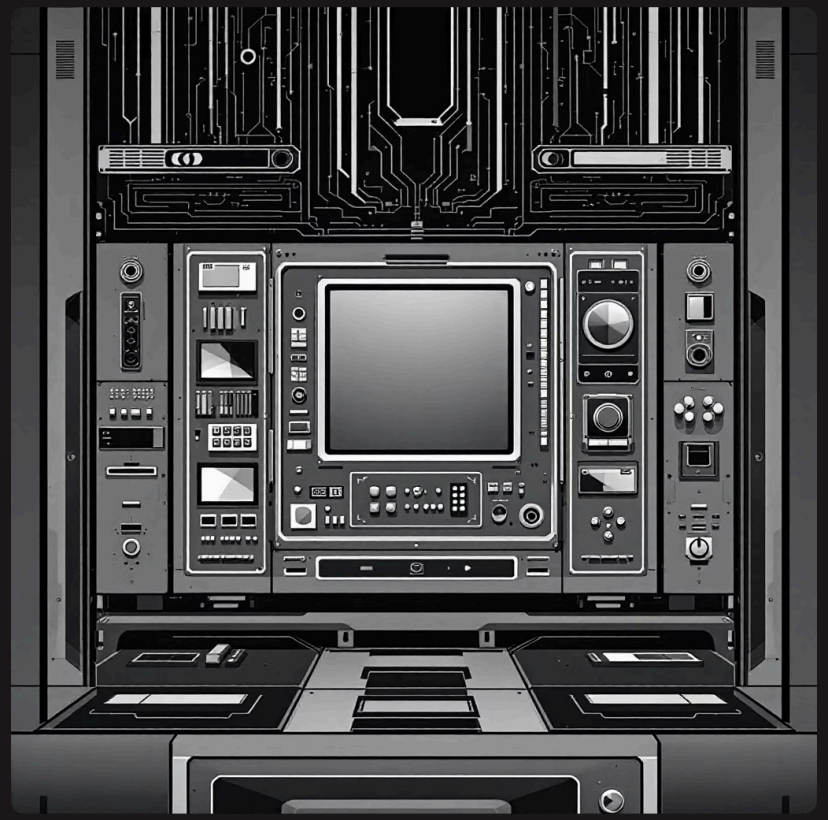
Facade

Exemplo aplicado ao varejo de bebidas

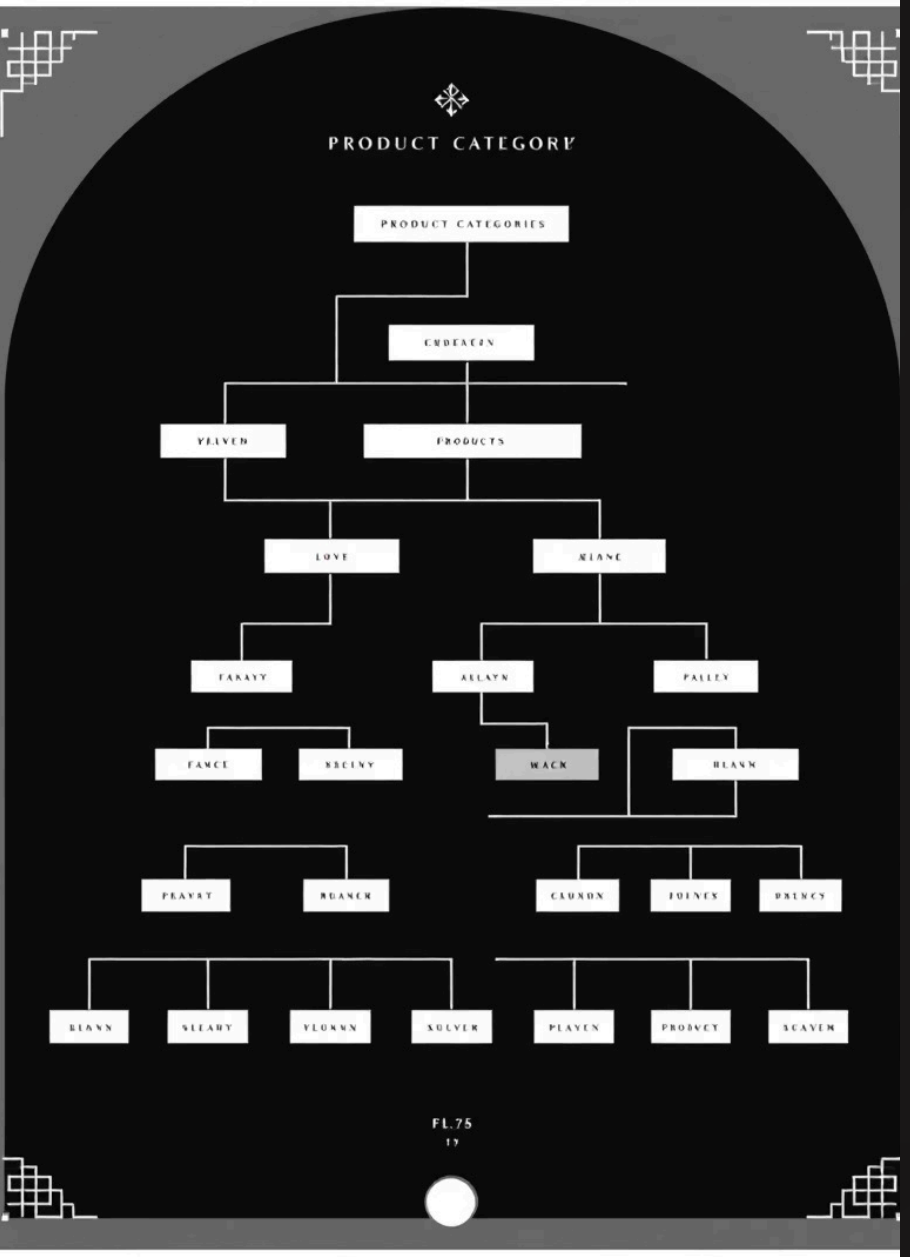
Contexto: Para registrar uma venda, o sistema precisa coordenar diversas ações: reduzir estoque, registrar itens de pedido, calcular totais, aplicar descontos, gerar transação de pagamento, atualizar status de pedido e emitir alertas relevantes.

Solução: Criar uma fachada, por exemplo `VendaFacade`, com métodos de alto nível como `realizarVenda()`, `consultarEstoque()` e `gerarRelatorioEstoque()` que orquestram as chamadas aos subsistemas.

Benefício: A aplicação cliente interage com uma única classe de entrada — sem conhecer detalhes internos de MongoDB, coleções específicas, cálculos complexos ou validações. Isso melhora a manutenibilidade e a clareza do código.



Made with GAMMA



Composite

Problema e Objetivo

Problema

O varejo organiza produtos por **categorias**, **subcategorias** e às vezes por famílias de produtos, formando estruturas hierárquicas. O sistema precisa tratar categorias simples e categorias compostas de modo uniforme.

Objetivo

Representar estruturas hierárquicas **parte-todo** (ex.: categoria → subcategoria → produto) permitindo manipular elementos individuais e composições de maneira uniforme, simplificando a lógica cliente.

Made with GAMMA

Composite

Exemplo aplicado ao varejo de bebidas

Contexto: Uma categoria "Destilados" pode conter "Vodka", "Whisky", "Tequila", e cada uma dessas contém diversos produtos específicos. A lógica de relatórios e filtros precisa funcionar tanto para um produto individual quanto para uma categoria inteira, incluindo suas subcategorias.

Solução: Criar interfaces comuns, como `ICategoria`, e classes como `CategoriaSimples` (para produtos finais) e `CategoriaComposta` (para agrupamentos). O cliente pode chamar métodos como `calcularEstoqueTotal()` ou `listarItens()` sem saber se está manipulando um item unitário ou um agrupamento complexo.

Benefício: Relatórios e consultas se tornam genéricos, permitindo navegar árvores de categorias de produtos sem a necessidade de códigos especiais ou condicionais complexos, facilitando a gestão e análise de dados.

Made with GAMMA

Encerramento

Integração dos padrões no sistema de PDV

Adapter

Resolve diferenças de formato entre fontes de dados diversas, padronizando a entrada no sistema.

Facade

Unifica o uso dos subsistemas mais complexos, oferecendo uma interface simplificada e de alto nível.

Composite

Organiza categorias e produtos em estruturas hierárquicas, permitindo tratamento uniforme de partes e agregados.

Esses três padrões estruturais são fundamentais para **reduzir o acoplamento**, tornar o código mais **claro e manutenível**, e ajudar a lidar com o **alto volume e a variedade de dados** do ambiente de varejo de bebidas. Eles promovem uma arquitetura robusta e escalável.

Made with GAMMA